GCA - Report 3

Maarten Roedolf

March 2024

1 Warps

```
__global__ void invert_image(uint8_t* image_array, uint8_t* new_image_array, int M, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int elementCount = M*N*C;

    int stride = blockDim.x * gridDim.x;

    for (int i = 0; i <= elementCount; i+=stride) {
        int idx = tid + i;
        new_image_array[idx] = 255 - image_array[idx];
    }
}
```

Figure 1: Inversion kernel code

I wrote a kernel to invert an image using its RGB-components. Each thread is responsible for a single component of a pixel and will therefore invert a single component. The kernel correctly works even when there are less threads than there are input elements thanks to the stride-loop.

```
void inversion_test(const char* input_path, const char* output_path, int M, int N) {
    clock_t start, end;

    uint8_t* h_image_array, *h_new_image_array, *d_image_array, *d_new_image_array;

    int total_timing = 0;
    for (int blockCount = 1; blockCount <= 4; blockCount++) {
        for (int threadCount = 1; threadCount <= 2048; threadCount*=2) {
            start = clock();
            // Read the image
            h_image_array = get_image_array(input_path, M, N);

            // Allocate output
            h_new_image_array = (uint8_t*)malloc(M*N*C);

            // Allocate memory on device

            cudaMalloc((void **)&d_image_array, M*N*C);
            cudaMalloc((void **)&d_new_image_array, M*N*C);

            // Copy input array to device
            cudaMemcpy(d_image_array, h_image_array, M*N*C, cudaMemcpyHostToDevice);
            cudaMemcpy(d_new_image_array, h_new_image_array, M*N*C, cudaMemcpyHostToDevice);
            invert_image<<<blockCount, threadCount>>>(d_image_array, d_new_image_array, M, N);
            cudaMemcpy(h_new_image_array, d_new_image_array, M*N*C, cudaMemcpyDeviceToHost);

            // Save the image
            end = clock();
            int elapsedCycles = end - start;
            save_image_array(h_new_image_array, output_path, M, N);
            std::cout << "Blocks: " << blockCount << "\tthreads per block: " << i << "\tcycles: " << elapsedCycles << std::endl;
            total_timing += elapsedCycles;
        }
    }

    std::cout << "Total cycles elapsed: " << total_timing << "\t average elapsed cycles: " << total_timing / (6*12) << std::endl;


}
```

The inversion test function will iterate over different thread configurations to assess which configurations are most efficiënt for which image. We omit the first kernel call from the average calculation because this call can be unreasonably longer than normal.

```
Starting Lenna test
Blocks: 1       threads per block: 1    cycles: 278
Blocks: 1       threads per block: 2    cycles: 31
Blocks: 1       threads per block: 4    cycles: 24
Blocks: 1       threads per block: 8    cycles: 16
Blocks: 1       threads per block: 16   cycles: 14
Blocks: 1       threads per block: 32   cycles: 12
Blocks: 1       threads per block: 64   cycles: 7
Blocks: 1       threads per block: 128  cycles: 4
Blocks: 1       threads per block: 256  cycles: 6
Blocks: 1       threads per block: 512  cycles: 4
Blocks: 1       threads per block: 1024 cycles: 4
Blocks: 1       threads per block: 2048 cycles: 2
Blocks: 2       threads per block: 1    cycles: 31
Blocks: 2       threads per block: 2    cycles: 21
Blocks: 2       threads per block: 4    cycles: 16
Blocks: 2       threads per block: 8    cycles: 13
Blocks: 2       threads per block: 16   cycles: 14
Blocks: 2       threads per block: 32   cycles: 7
Blocks: 2       threads per block: 64   cycles: 7
Blocks: 2       threads per block: 128  cycles: 5
Blocks: 2       threads per block: 256  cycles: 4
Blocks: 2       threads per block: 512  cycles: 3
Blocks: 2       threads per block: 1024 cycles: 5
Blocks: 2       threads per block: 2048 cycles: 3
Blocks: 3       threads per block: 1    cycles: 23
Blocks: 3       threads per block: 2    cycles: 16
Blocks: 3       threads per block: 4    cycles: 13
Blocks: 3       threads per block: 8    cycles: 12
Blocks: 3       threads per block: 16   cycles: 7
Blocks: 3       threads per block: 32   cycles: 6
Blocks: 3       threads per block: 64   cycles: 6
Blocks: 3       threads per block: 128  cycles: 4
Blocks: 3       threads per block: 256  cycles: 5
Blocks: 3       threads per block: 512  cycles: 3
Blocks: 3       threads per block: 1024 cycles: 5
Blocks: 3       threads per block: 2048 cycles: 4
Blocks: 4       threads per block: 1    cycles: 22
Blocks: 4       threads per block: 2    cycles: 17
Blocks: 4       threads per block: 4    cycles: 12
Blocks: 4       threads per block: 8    cycles: 13
Blocks: 4       threads per block: 16   cycles: 9
Blocks: 4       threads per block: 32   cycles: 6
Blocks: 4       threads per block: 64   cycles: 4
Blocks: 4       threads per block: 128  cycles: 3
Blocks: 4       threads per block: 256  cycles: 5
Blocks: 4       threads per block: 512  cycles: 5
Blocks: 4       threads per block: 1024 cycles: 5
Blocks: 4       threads per block: 2048 cycles: 3
Total cycles elapsed: 462       average elapsed cycles: 6
```

(a) Lenna Image Test Results

```
Starting VR test
Blocks: 1       threads per block: 1    cycles: 123
Blocks: 1       threads per block: 2    cycles: 73
Blocks: 1       threads per block: 4    cycles: 47
Blocks: 1       threads per block: 8    cycles: 34
Blocks: 1       threads per block: 16   cycles: 28
Blocks: 1       threads per block: 32   cycles: 25
Blocks: 1       threads per block: 64   cycles: 17
Blocks: 1       threads per block: 128  cycles: 10
Blocks: 1       threads per block: 256  cycles: 7
Blocks: 1       threads per block: 512  cycles: 5
Blocks: 1       threads per block: 1024 cycles: 7
Blocks: 1       threads per block: 2048 cycles: 5
Blocks: 2       threads per block: 1    cycles: 74
Blocks: 2       threads per block: 2    cycles: 48
Blocks: 2       threads per block: 4    cycles: 34
Blocks: 2       threads per block: 8    cycles: 28
Blocks: 2       threads per block: 16   cycles: 26
Blocks: 2       threads per block: 32   cycles: 14
Blocks: 2       threads per block: 64   cycles: 9
Blocks: 2       threads per block: 128  cycles: 7
Blocks: 2       threads per block: 256  cycles: 6
Blocks: 2       threads per block: 512  cycles: 5
Blocks: 2       threads per block: 1024 cycles: 6
Blocks: 2       threads per block: 2048 cycles: 6
Blocks: 3       threads per block: 1    cycles: 57
Blocks: 3       threads per block: 2    cycles: 41
Blocks: 3       threads per block: 4    cycles: 29
Blocks: 3       threads per block: 8    cycles: 25
Blocks: 3       threads per block: 16   cycles: 18
Blocks: 3       threads per block: 32   cycles: 12
Blocks: 3       threads per block: 64   cycles: 8
Blocks: 3       threads per block: 128  cycles: 9
Blocks: 3       threads per block: 256  cycles: 10
Blocks: 3       threads per block: 512  cycles: 7
Blocks: 3       threads per block: 1024 cycles: 9
Blocks: 3       threads per block: 2048 cycles: 6
Blocks: 4       threads per block: 1    cycles: 48
Blocks: 4       threads per block: 2    cycles: 35
Blocks: 4       threads per block: 4    cycles: 30
Blocks: 4       threads per block: 8    cycles: 27
Blocks: 4       threads per block: 16   cycles: 18
Blocks: 4       threads per block: 32   cycles: 11
Blocks: 4       threads per block: 64   cycles: 8
Blocks: 4       threads per block: 128  cycles: 9
Blocks: 4       threads per block: 256  cycles: 12
Blocks: 4       threads per block: 512  cycles: 8
Blocks: 4       threads per block: 1024 cycles: 8
Blocks: 4       threads per block: 2048 cycles: 5
Total cycles elapsed: 972       average elapsed cycles: 13
```

(b) VR Image Test Results

Figure 2: Image inversion tests

We see in the test results that the VR image takes 10 cycles longer on average than the Lenna image. This can partly be explained due to the fact that the image has more pixels, but is also causes by the fact that the dimensions are multiples of 32 (the size of warps) which is not the case in the VR image.



(a) Inverted Lenna Image



(b) Inverted VR Image

Figure 3: Image inversion results

2 Divergence

```
void divergence_test(const char* input_path, const char* output_path, int M, int N) {
    clock_t start, end;

    uint8_t* h_image_array, *h_new_image_array, *d_image_array, *d_new_image_array;

    int total_timing = 0;
    for (int blockCount = 1; blockCount <= 4; blockCount++) {
        for (int threadCount = 1; threadCount <= 2048; threadCount*=2) {
            start = clock();
            // Read the image
            h_image_array = get_image_array(input_path, M, N);

            // Allocate output
            h_new_image_array = (uint8_t*)malloc(M*N*C);

            // Allocate memory on device

            cudaMalloc((void **)&d_image_array, M*N*C);
            cudaMalloc((void **)&d_new_image_array, M*N*C);

            // Copy input array to device
            cudaMemcpy(d_image_array, h_image_array, M*N*C, cudaMemcpyHostToDevice);
            cudaMemcpy(d_new_image_array, h_new_image_array, M*N*C, cudaMemcpyHostToDevice);

            invert_image_divergence<<<blockCount, threadCount>>>(d_image_array, d_new_image_array, M, N);

            cudaMemcpy(h_new_image_array, d_new_image_array, M*N*C, cudaMemcpyDeviceToHost);

            // Save the image
            end = clock();
            int elapsedCycles = end - start;
            save_image_array(h_new_image_array, output_path, M, N);
            std::cout << "Blocks: " << blockCount << "\tthreads per block: " << threadCount << "\tcycles: " << elapsedCycles << std::endl;
            // First call takes longer so omit from average calc
            if (total_timing == 0)total_timing = 1;
            else total_timing += elapsedCycles;
        }
    }

    std::cout << "Total cycles elapsed: " << total_timing << "\t average elapsed cycles: " << total_timing / ((6*12)-1) << std::endl;
}
```

Figure 4: Divergence Kernel Code

The kernel which causes thread divergence is mostly the same as the previous kernel but with added complexity to the threads that are assigned to the R-component.

```
void divergence_test(const char* input_path, const char* output_path, int M, int N) {
    clock_t start, end;

    uint8_t* h_image_array, *h_new_image_array, *d_image_array, *d_new_image_array;

    int total_timing = 0;
    for (int blockCount = 1; blockCount <= 4; blockCount++) {
        for (int threadCount = 1; threadCount <= 2048; threadCount*=2) {
            start = clock();
            // Read the image
            h_image_array = get_image_array(input_path, M, N);

            // Allocate output
            h_new_image_array = (uint8_t*)malloc(M*N*C);

            // Allocate memory on device

            cudaMalloc((void **)&d_image_array, M*N*C);
            cudaMalloc((void **)&d_new_image_array, M*N*C);

            // Copy input array to device
            cudaMemcpy(d_image_array, h_image_array, M*N*C, cudaMemcpyHostToDevice);
            cudaMemcpy(d_new_image_array, h_new_image_array, M*N*C, cudaMemcpyHostToDevice);

            invert_image_divergence<<<blockCount, threadCount>>>(d_image_array, d_new_image_array, M, N);

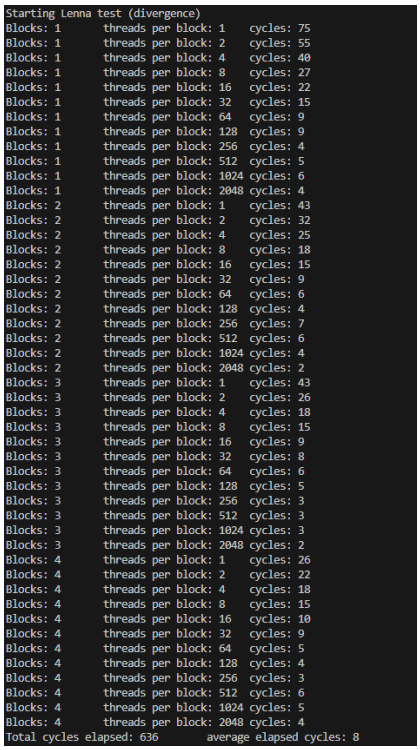
            cudaMemcpy(h_new_image_array, d_new_image_array, M*N*C, cudaMemcpyDeviceToHost);

            // Save the image
            end = clock();
            int elapsedCycles = end - start;
            save_image_array(h_new_image_array, output_path, M, N);
            std::cout << "Blocks: " << blockCount << "\tthreads per block: " << threadCount << "\tcycles: " << elapsedCycles << std::endl;
            // First call takes longer so omit from average calc
            if (total_timing == 0)total_timing = 1;
            else total_timing += elapsedCycles;
        }
    }

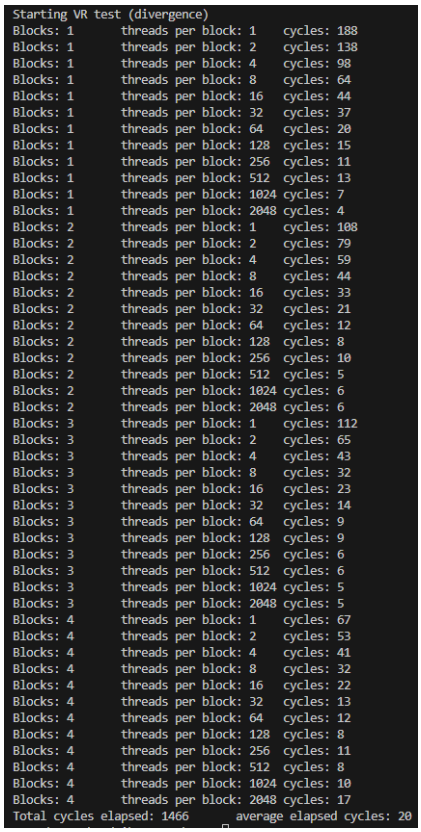
    std::cout << "Total cycles elapsed: " << total_timing << "\t average elapsed cycles: " << total_timing / ((6*12)-1) << std::endl;
}
```

Figure 5: Divergence Test Code

The divergence test code is the same as the previous test code with the only difference being that I call the divergence kernel.



(a) Lenna Image (Divergence) Test Results



(b) VR Image Test Results

Figure 6: Image inversion tests with divergence

The test results show as that in both cases the computation time increases quite a lot when divergence happens.



(a) Lenna Image (b) VR Image

Figure 7: Inverted images with divergence